

1. LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos especifica as condições, funções e características que o Sistema de Gestão de Projetos e Equipes deve atender para suprir a necessidade de negócio apresentada.

1.1 Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as ações, comportamentos e informações que o sistema deve executar e gerenciar.

- **[RF01] Controle de Primeiro Acesso:** O sistema deve verificar, logo na inicialização, se existem usuários administradores cadastrados no banco de dados. Caso não existam, o sistema deve direcionar o usuário obrigatoriamente para a tela de primeiro cadastro; caso contrário, deve exibir a tela de login.
- **[RF02] Autenticação e Controle de Acesso (Login):** O sistema deve validar as credenciais de acesso (login e senha) dos usuários e restringir as funcionalidades de acordo com o perfil logado (Administrador, Gerente ou Colaborador).
- **[RF03] Cadastro e Manutenção de Usuários:** O sistema deve permitir a inserção, consulta, atualização e exclusão (CRUD) de usuários, armazenando: Nome completo, CPF, E-mail, Cargo, Login, Senha e Perfil de acesso.
- **[RF04] Cadastro e Manutenção de Projetos:** O sistema deve permitir o gerenciamento de projetos, contendo: Nome do projeto, Descrição, Data de início, Data de término prevista, Status (Planejado, Em Andamento, Concluído, Cancelado) e a associação de um Gerente Responsável.
- **[RF05] Cadastro e Composição de Equipes:** O sistema deve permitir a criação de equipes (Nome e Descrição) e vincular múltiplos usuários (Colaboradores) a essas equipes.
- **[RF06] Alocação de Equipes em Projetos:** O sistema deve permitir que uma equipe seja associada e alocada em um ou mais projetos vigentes.
- **[RF07] Emissão de Relatórios de Acompanhamento:** O sistema deve gerar relatórios consolidados que demonstrem o status atual dos projetos e o desempenho/alocação das equipes.

1.2 Requisitos Não-Funcionais (RNF)

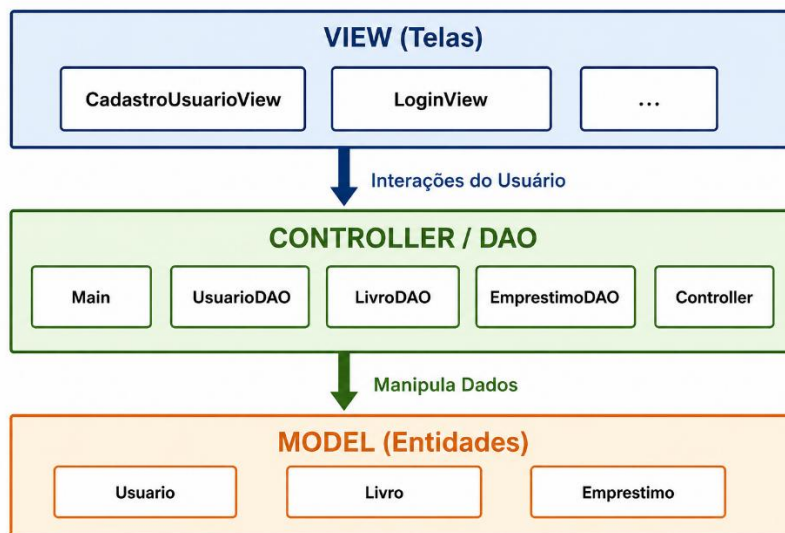
Os requisitos não-funcionais determinam os critérios tecnológicos, de segurança e de usabilidade do software.

- **[RNF01] Linguagem de Programação:** O software deve ser desenvolvido utilizando a linguagem de programação Java.
- **[RNF02] Persistência de Dados:** O sistema deve utilizar um Sistema Gerenciador de Banco de Dados Relacional (SGBD), utilizando a linguagem SQL para manipulação dos dados.
- **[RNF03] Interface Gráfica:** A interação com o usuário deve ocorrer por meio de uma Interface Gráfica de Usuário (GUI) desktop estável e intuitiva.

- **[RNF04] Arquitetura e Padrões de Projeto:** O código-fonte deve ser estruturado seguindo o padrão de arquitetura MVC (Model-View-Controller) aliado ao padrão DAO (Data Access Object) para isolamento das instruções SQL.

2. MODELAGEM DO SISTEMA (POO + MVC)

O sistema foi modelado aplicando os pilares da Programação Orientada a Objetos (POO) — como encapsulamento, herança e polimorfismo — estruturado sob o padrão arquitetural MVC, garantindo a separação de responsabilidades e facilitando a manutenção do código.



2.1 Camada Model (Camada de Negócio e Dados)

Contém as classes que representam as entidades do mundo real. Os atributos são estritamente protegidos por modificadores de acesso privados (`private`), sendo acessados externamente apenas por métodos públicos de leitura e escrita (`getters` e `setters`).

- **Classe Usuario:** Representa o corpo de colaboradores e gestores da empresa. Possui atributos encapsulados como `id`, `nomeCompleto`, `cpf`, `email`, `cargo`, `login`, `senha` e `perfil`.
- **Classe Projeto:** Modela as demandas de trabalho entregues pela Oracle. Possui atributos como `id`, `nomeProjeto`, `descricao`, `dataInicio`, `dataTerminoPrevista`, `status` e uma referência ao `Usuario` (do perfil `Gerente`) responsável pela entrega.
- **Classe Equipe:** Define o agrupamento de funcionários. Possui os atributos `id`, `nomeEquipe`, `descricao` e uma estrutura de dados (como uma `List<Usuario>`) para armazenar os membros vinculados.

2.2 Camada View (Camada de Interface)

Responsável por renderizar os componentes gráficos na tela do usuário e capturar eventos (como o clique de botões). Não processa regras de negócio e não se comunica diretamente com a base de dados SQL.

- **Classe CadastroUsuarioView:** Janela gráfica utilizada no fluxo de primeiro acesso do sistema ou por administradores para registrar novas contas.
- **Classe LoginView:** Janela inicial de autenticação que captura o login e a senha digitados pelo usuário.

- **Classes Complementares de View** (ProjetoView, EquipeView): Telas com formulários, tabelas e menus para a manipulação visual dos dados do sistema.

2.3 Camada DAO / Controller (Camada de Controle e Persistência)

Atua como a ponte entre as interações vindas da **View** e as operações na camada **Model**, isolando toda a lógica de persistência e conexões JDBC em classes especializadas (Data Access Object).

- **Classe Main:** Ponto de entrada do sistema (public static void main). É responsável por inicializar o fluxo lógico principal, instanciando o UsuarioDAO para validar o estado do banco e decidir se o sistema deve exibir a tela de login ou a tela de primeiro cadastro.
- **Classe UsuarioDAO:** Concentra os métodos estruturados de persistência de dados dos usuários (instruções SQL). Executa funções como:
 - existeUsuario(): Realiza uma varredura (SELECT COUNT) na base de dados para checar a existência de registros.
 - cadastrar(Usuario usuario): Recebe um objeto do tipo Model e realiza o INSERT na tabela correspondente.
 - login(String login, String senha): Realiza a consulta de autenticação retornando um valor lógico (boolean) de sucesso.
- **Classes ProjetoDAO e EquipeDAO:** Responsáveis por gerenciar as rotinas de criação, leitura, atualização e exclusão (CRUD) dos dados de projetos, equipes e suas respectivas alocações no banco de dados relacional.